
AN0050- EC NB-IoT GPIO 应用笔记_V1.0

EigenCOMM Wireless Microcontroller

文档说明

本文档描述移芯通信 NB-IoT 芯片的 GPIO 引脚的配置使用，以及使用时的注意事项。
本文旨在帮助客户在开发过程中，快速的完成基于移芯通信 NB-IoT 芯片的 GPIO 引脚与其他设备的连接使用。

目录

1. 引言.....	3
1.1 GPIO 简介	3
1.2 文档目的.....	3
2. 说明.....	4
2.1 AON_GPIO 与普通 GPIO 的区别.....	4
2.2 AON_GPIO 与普通 GPIO 的电平选择.....	4
2.2.1 普通 GPIO 的电平选择	4
2.2.2 AON_GPIO 的电平选择	4
2.3 示例代码.....	4
3. 注意事项.....	7
3.1 配置两个 Instance 的 GPIO 输入中断遇到的问题	7
4. 版本.....	9
5. 关于我们.....	10

1. 引言

1.1 GPIO 简介

GPIO (General Purpose I/O Ports) 意思为通用输入/输出端口, 通俗地说, 就是移芯 NB-IOT 芯片可控制的一些引脚, 可以通过它们输出高低电平或者通过它们读入引脚的状态-是高电平或是低电平。GPIO 口一是个比较重要的概念, 用户可以通过 GPIO 口和硬件进行数据交互 (如 UART), 控制硬件工作 (如 LED、蜂鸣器等), 读取硬件的工作状态信号 (如中断信号) 等。

1.2 文档目的

介绍移芯 NB-IOT 芯片 EC616/EC616S 的 GPIO 和 AON_GPIO 的在 ACTIVE 和 IDLE 状态下是如何使用的。AON_GPIO 是 always on GPIO 的简称, 是 EC616 所特有的一类 GPIO。SLEEP1、SLEEP2 和 HIBERNATE 下 AON_GPIO 的使用请参考 AN0024-EC NB-IoT PMU 低功耗开发手册, 具体可参考 PLAT\project\ec616_0h00\apps\slpman_example 低功耗例程。

说明: AON_GPIO 仅仅在 EC616 上才有, EC616S 不存在 AON_GPIO

2. 说明

2.1 AON_GPIO 与普通 GPIO 的区别

AON GPIO 能够在深度睡眠（SLEEP1, SLEEP2, HIBERNATE）的时候，仍保持有效输出。而 GPIO 在深度休眠的时候，是掉电的。ACTIVE 和 IDLE 状态下 AON_GPIO 与普通 GPIO 使用一样，支持输出、输入及外部中断，只要调用 `slpManAONIOPowerOn()`；使能 AON_GPIO 的电源。后面 API 调用两者共用。

2.2 AON_GPIO 与普通 GPIO 的电平选择

2.2.1 普通 GPIO 的电平选择

有两种：

- (1) 硬件 IO_1833_SEL 引脚选择：浮空为 1.8V，接地为 3.3V。
- (2) 软件调节 IO 电平，API 是 `void slpManNormalIOVltSet(IOVoltageSel_t sel)`。当硬件选择 1.8V 时，软件可调节 IO 电压范围为 1.65V~3.00V；当硬件选择 3.3V 时，软件可调节 IO 电压范围为 2.65V~3.40V。

注意：

- (a) 软件没有调节时，以硬件选择的电压为准。IO 的电压不会超过 VBAT 的电压。
- (b) 在睡眠后，普通 IO 的电平会根据硬件选择，及软件配置。恢复为 1.8V/2.8V/3.2V。

如原本选择 1.75V，则唤醒后恢复成 1.8V。如原本选择 3.4V，则唤醒后恢复成 3.2V。

为保证电平准确的恢复到预期值：

对于 sleep1，需要注册唤醒后回调函数，重新配置 normal io 电平。

对于 sleep2/hibernate，运行到 Bsp_CustomInit 后重新配置 normal io 电压。

2.2.2 AON_GPIO 的电平选择

调用 `slpManAONIOPowerOn()` 接口给 AON_GPIO 上电后，默认电压是 1.8V。然后可以调用 API `void slpManAONIOVltSet(IOVoltageSel_t sel)` 调节电压范围为 1.65V~3.40V。

注意：IO 的电压不会超过 VBAT 的电压。

2.3 示例代码

可结合 `PLAT\project\ec616_0h00\apps\driver_example\src\gpio_demo.c`

以下代码是设置 AON_GPIO2，步骤如下：

- (1) 使能 AON_GPIO 电源 `slpManAONIOPowerOn()`，普通 GPIO 不需要。
- (2) 查找 EC616 数据手册中管脚复用表格 AON_GPIO2 对应 GPIO21。

AON GPIO	GPIO20 AON1	31	I, PD	GPIO20
	GPIO21 AON2	32	I, PD	GPIO21
	GPIO22 AON3	33	I, PD	GPIO22
	GPIO23 AON4	34	I, PD	GPIO23
	GPIO24 AON5	35	I, PD	GPIO24
	GPIO25 AON6	36	I, PD	GPIO25

(3) 调用 PAD 的 API 配置管脚复用为 GPIO

```
void PAD_SetPinConfig(uint32_t pin, const pad_config_t *config)
```

参数说明: pin 值对应管脚复用表格 paddr 那一列的数值就是 32。故代码为 PAD_SetPinConfig(32, &padConfig)。(普通 GPIO 与此类似)。Padconfig 则可以根据需要, 配置内部输入上下拉等。

(4) 配置 GPIO 具体功能 API

```
void GPIO_PinConfig(uint32_t port, uint16_t pin, const gpio_pin_config_t *config)
```

参数说明: 此时要确定 port 和 pin 的值, 其中 port 和 pin 按照如下的算法确定

$port = Index / 16$

$pin = Index \% 16$

AON2 属于 GPIO21, 所以 Index=21, $port=21/16=1$, $pin=21\%16=5$, 所以最终的接口为:

```
GPIO_PinConfig(1, 5, &config)
```

config 则可以配置 GPIO 是输入还是输出, 输出高电平或低电平, 输入中断是否开启, 中断是边沿还是电平触发等。

说明: 在 pinmux 表中, GPIO 的索引从 0 到最大 25, 但是一个 GPIO Instance 最多只能控制 16 个 GPIO。其中 GPIO0~GPIO15 属于 Instance0, GPIO16~GPIO25 属于 Instance1。

```
slpManAONIOPowerOn();//使能 AON 电源
```

```
/* PAD 设置*/
```

```
pad_config_t padConfig;
```

```
PAD_GetDefaultConfig(&padConfig);
```

```
padConfig.mux = PAD_MuxAlt0;
```

```
padConfig.pullSelect = PAD_PullInternal;
```

```
padConfig.pullDownEnable = PAD_PullDownDisable;//如果外部上拉, 内部下拉要去掉
```

```
padConfig.pullUpEnable = PAD_PullUpDisable; //AON GPIO 没有 PullUp
```

```
PAD_SetPinConfig(32, &padConfig);
```

/* GPIO 设置*/

gpio_pin_config_t config;

config.pinDirection = GPIO_DirectionInput;

config.misc.interruptConfig = GPIO_InterruptFallingEdge;//配置下降沿触发

GPIO_PinConfig(1, 5, &config);

3. 注意事项

3.1 配置两个 Instance 的 GPIO 输入中断遇到的问题

当快速频繁触发两个中断，会出现其中一个 GPIO 中断进入中断函数后，在调用 GPIO_SaveAndSetIRQMask 关闭两个 Instance 的中断后，但是另一个 GPIO 所属 Instance 的 INTSTATUS 寄存器还是被置位的情况，如果这个仍被置位的 INTSTATUS 寄存器未被 clear，会导致后面不产生中断，不再进入中断处理函数。

问题代码：

```
64: */
65: void GPIO_ISR()
66: {
67:     printf("start GPIO17 %x\r\n", GPIO_GetInterruptFlags(BUTTON_GPIO_INSTANCE));
68:     printf("start GPIO7 %x\r\n", GPIO_GetInterruptFlags(LED_GPIO_INSTANCE));
69:
70:     //Save current irq mask and diable whole port interrupts to get rid of interrupt overflow
71:     uint16_t portIrqMask = GPIO_SaveAndSetIRQMask(BUTTON_GPIO_INSTANCE);
72:     uint16_t portIrqMask_0 = GPIO_SaveAndSetIRQMask(LED_GPIO_INSTANCE);
73:
74:     printf("before GPIO17 %x, %x\r\n", portIrqMask, GPIO_GetInterruptFlags(BUTTON_GPIO_INSTANCE));
75:     printf("before GPIO7 %x, %x\r\n", portIrqMask_0, GPIO_GetInterruptFlags(LED_GPIO_INSTANCE));
76:
77:     if (GPIO_GetInterruptFlags(BUTTON_GPIO_INSTANCE) & (1 << BUTTON_GPIO_PIN))
78:     {
79:         gpioInterruptCount++;
80:         GPIO_ClearInterruptFlags(BUTTON_GPIO_INSTANCE, 1 << BUTTON_GPIO_PIN);
81:     }
82:
83:     else if (GPIO_GetInterruptFlags(LED_GPIO_INSTANCE) & (1 << LED_GPIO_PIN))
84:     {
85:         gpioInterruptCount++;
86:         GPIO_ClearInterruptFlags(LED_GPIO_INSTANCE, 1 << LED_GPIO_PIN);
87:     }
88:
89:     printf("middle GPIO17 %x, %x\r\n", portIrqMask, GPIO_GetInterruptFlags(BUTTON_GPIO_INSTANCE));
90:     printf("middle GPIO7 %x, %x\r\n", portIrqMask_0, GPIO_GetInterruptFlags(LED_GPIO_INSTANCE));
91:
92:     GPIO_RestoreIRQMask(BUTTON_GPIO_INSTANCE, portIrqMask);
93:     GPIO_RestoreIRQMask(LED_GPIO_INSTANCE, portIrqMask_0);
94:
95:     printf("after GPIO17 %x, %x\r\n", portIrqMask, GPIO_GetInterruptFlags(BUTTON_GPIO_INSTANCE));
96:     printf("after GPIO7 %x, %x\r\n", portIrqMask_0, GPIO_GetInterruptFlags(LED_GPIO_INSTANCE));
97:
98: } // end GPIO_ISR
99:
```

规避该问题代码，方案是判断所有 GPIO 输入中断 INTSTATUS 状态，若有效，都清掉：

```

65: void GPIO_ISR()
66: {
67:     printf("start GPIO17 %x\r\n", GPIO_GetInterruptFlags(BUTTON_GPIO_INSTANCE));
68:     printf("start GPIO7 %x\r\n", GPIO_GetInterruptFlags(LED_GPIO_INSTANCE));
69:
70:     //Save current irq mask and diable whole port interrupts to get rid of interrupt overflow
71:     uint16_t portIrqMask = GPIO_SaveAndSetIRQMask(BUTTON_GPIO_INSTANCE);
72:     uint16_t portIrqMask_0 = GPIO_SaveAndSetIRQMask(LED_GPIO_INSTANCE);
73:
74:     printf("before GPIO17 %x, %x\r\n", portIrqMask, GPIO_GetInterruptFlags(BUTTON_GPIO_INSTANCE));
75:     printf("before GPIO7 %x, %x\r\n", portIrqMask_0, GPIO_GetInterruptFlags(LED_GPIO_INSTANCE));
76:
77:     if (GPIO_GetInterruptFlags(BUTTON_GPIO_INSTANCE) & (1 << BUTTON_GPIO_PIN))
78:     {
79:         gpioInterruptCount++;
80:         GPIO_ClearInterruptFlags(BUTTON_GPIO_INSTANCE, 1 << BUTTON_GPIO_PIN);
81:     }
82:
83:     if (GPIO_GetInterruptFlags(LED_GPIO_INSTANCE) & (1 << LED_GPIO_PIN))
84:     {
85:         gpioInterruptCount++;
86:         GPIO_ClearInterruptFlags(LED_GPIO_INSTANCE, 1 << LED_GPIO_PIN);
87:     }
88:
89:     printf("middle GPIO17 %x, %x\r\n", portIrqMask, GPIO_GetInterruptFlags(BUTTON_GPIO_INSTANCE));
90:     printf("middle GPIO7 %x, %x\r\n", portIrqMask_0, GPIO_GetInterruptFlags(LED_GPIO_INSTANCE));
91:
92:     GPIO_RestoreIRQMask(BUTTON_GPIO_INSTANCE, portIrqMask);
93:     GPIO_RestoreIRQMask(LED_GPIO_INSTANCE, portIrqMask_0);
94:
95:     printf("after GPIO17 %x, %x\r\n", portIrqMask, GPIO_GetInterruptFlags(BUTTON_GPIO_INSTANCE));
96:     printf("after GPIO7 %x, %x\r\n", portIrqMask_0, GPIO_GetInterruptFlags(LED_GPIO_INSTANCE));
97: } « end GPIO_ISR »
98:

```


4. 版本

版本	日期	备注
Draft	2021-06-07	

5. 关于我们

上海移芯通信科技有限公司坐落于上海张江，公司于 2017 年 2 月成立，从事蜂窝移动通信芯片的研发和销售。公司产品主要应用于广域物联网通信领域，致力于设计最具性价比的蜂窝物联网芯片。公司团队在蜂窝通信芯片上有着辉煌历史和丰富经验。公司所有核心技术全部自研，包括算法&架构、射频、基带、SoC、协议栈软件、平台&应用软件和硬件方案等。

移芯通信首款自主研发的超低功耗 NB-IoT 芯片 EC616 于 2019 年中量产，被绝大部分头部模组企业采用，现已大批量应用于全国各区域各行业。下一代超高集成度 NB-IoT 芯片 EC616S 于 20 年底量产，超低功耗 4G Cat.1 芯片 EC618 工程样片阶段，5G 芯片正在研发中。

上海移芯通信科技有限公司

地址：中国上海市浦东新区祥科路 298 号佑越国际 1 幢 7 层

邮编：201210

电话：

电邮：

网址：<http://www.eigencomm.com>

技术支持窗口

电邮：support@eigencomm.com